EXPNO:5	CLUSTERING WITH K-MEANS AND DIMENSIONALITY REDUCTION WITH PCA

AIM:

To demonstrate the application of Unsupervised Learning models, specifically K-Means clustering for grouping data points and Principal Component Analysis (PCA) for dimensionality reduction and visualization, using a suitable dataset.

ALGORITHM:

1. K-Means Clustering

K-Means is an iterative clustering algorithm that aims to partition n observations into k clusters, where each observation belongs to the cluster with the nearest mean (centroid).

Steps:

1. Initialization: Choose k initial centroids randomly from the dataset.
2. Assignment: Assign each data point to the cluster whose centroid is closest (e.g., using
1. Euclidean distance).
2. Update: Recalculate the centroids as the mean of all data points assigned to that cluster.
3. Iteration: Repeat steps 2 and 3 until the centroids no longer move significantly or a
4. maximum number of iterations is reached.

2. Principal Component Analysis (PCA)

PCA is a statistical procedure that uses an orthogonal transformation to convert a set of observations of possibly correlated variables into a set of values of linearly uncorrelated variables called principal components.

Steps:

1. Standardization: Standardize the dataset (mean = 0, variance = 1).
2. Covariance Matrix Calculation: Compute the covariance matrix of the standardized data.
3. Eigenvalue Decomposition: Calculate the eigenvalues and eigenvectors of the covariance
1. matrix.
4. Feature Vector Creation: Sort the eigenvectors by decreasing eigenvalues and select the
2. top k eigenvectors to form a feature vector (projection matrix).
5. Projection: Project the original data onto the new feature space using the feature vector.

CODE:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.datasets import make_blobs

from sklearn.preprocessing import StandardScaler

from sklearn.cluster import KMeans

from sklearn.decomposition import PCA

from sklearn.metrics import silhouette_score

print("--- Part 1: K-Means Clustering ---")

X, y = make_blobs(

    n_samples=300,

    centers=3,

    cluster_std=0.60,

    random_state=42
```

```
)
df_kmeans = pd.DataFrame(
    X,
    columns=['Feature_1', 'Feature_2']
)
print("\nOriginal K-Means Dataset Head:")
print(df_kmeans.head())
wcss = []
for i in range(1, 11):
    kmeans = KMeans(
        n_clusters=i,
        init='k-means++',
        max_iter=300,
        n_init=10,
        random_state=42
    )
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)
plt.figure(figsize=(10, 6))
plt.plot(
    range(1, 11),
    wcss,
```

```
    marker='o',
    linestyle='--'
)
plt.title('Elbow Method for Optimal K (K-Means)')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS')
plt.grid(True)
plt.show()
optimal_k = 3
kmeans = KMeans(
    n_clusters=optimal_k,
    init='k-means++',
    max_iter=300,
    n_init=10,
    random_state=42
)
clusters = kmeans.fit_predict(X)
df_kmeans['Cluster'] = clusters
plt.figure(figsize=(10, 8))
sns.scatterplot(
    x='Feature_1',
    y='Feature_2',
```

```
    hue='Cluster',
    data=df_kmeans,
    palette='viridis',
    s=100,
    alpha=0.8
)
plt.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    s=300,
    c='red',
    marker='X',
    label='Centroids'
)
plt.title(f'K-Means Clustering with K={optimal_k}')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.grid(True)
plt.show()
silhouette_avg = silhouette_score(X, clusters)
print(
```

```
    f"\nSilhouette Score for K-Means "
    f"(K={optimal_k}): {silhouette_avg:.3f}"
)
print("\n--- Part 2: Dimensionality Reduction with PCA ---")
X_pca, y_pca = make_blobs(
    n_samples=500,
    n_features=4,
    centers=4,
    cluster_std=1.0,
    random_state=25
)
df_pca_original = pd.DataFrame(
    X_pca,
    columns=[f'Feature_{i+1}' for i in range(X_pca.shape[1])]
)
df_pca_original['True_Cluster'] = y_pca
print("\nOriginal PCA Dataset Head:")
print(df_pca_original.head())
print(
    f"Original PCA Dataset Shape: "
    f"{df_pca_original.shape}"
)
```

```
scaler = StandardScaler()
X_pca_scaled = scaler.fit_transform(X_pca)
pca = PCA(n_components=2)
principal_components = pca.fit_transform(
    X_pca_scaled
)
df_principal_components = pd.DataFrame(
    principal_components,
    columns=[
        'Principal_Component_1',
        'Principal_Component_2'
    ]
)
df_principal_components['True_Cluster'] = y_pca
explained_variance = pca.explained_variance_ratio_
print("\nPrincipal Components Head:")
print(df_principal_components.head())
print(
    f"\nExplained Variance Ratio: "
    f"{explained_variance}"
)
```

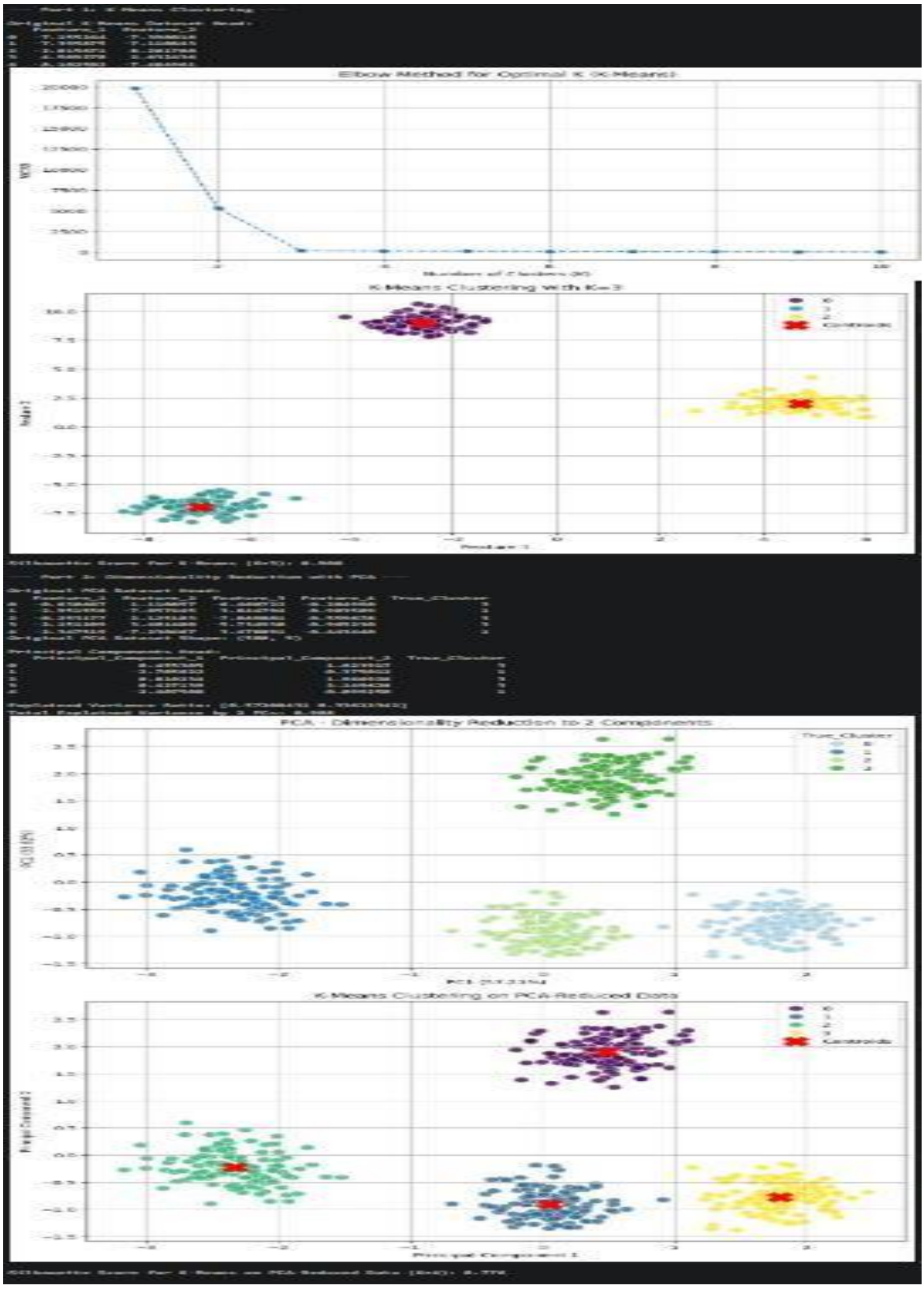
```
print(
    f"Total Explained Variance by 2 PCs: "
    f"{explained_variance.sum():.3f}"
)
plt.figure(figsize=(10, 8))
sns.scatterplot(
    x='Principal_Component_1',
    y='Principal_Component_2',
    hue='True_Cluster',
    data=df_principal_components,
    palette='Paired',
    s=100,
    alpha=0.8
)
plt.title(
    'PCA - Dimensionality Reduction to 2 Components'
)
plt.xlabel(
    f'PC1 ({explained_variance[0] * 100:.2f}%)'
)
plt.ylabel(
    f'PC2 ({explained_variance[1] * 100:.2f}%)'
```

```
)
plt.grid(True)
plt.show()
kmeans_pca = KMeans(
    n_clusters=4,
    init='k-means++',
    max_iter=300,
    n_init=10,
    random_state=42
)
clusters_pca = kmeans_pca.fit_predict(
    principal_components
)
df_principal_components[
    'KMeans_Cluster_on_PCA'
] = clusters_pca
plt.figure(figsize=(10, 8))
sns.scatterplot(
    x='Principal_Component_1',
    y='Principal_Component_2',
    hue='KMeans_Cluster_on_PCA',
    data=df_principal_components,
```

```
    palette='viridis',
    s=100,
    alpha=0.8
)
plt.scatter(
    kmeans_pca.cluster_centers_[:,  0],
    kmeans_pca.cluster_centers_[:, 1],
    s=300,
    c='red',
    marker='X',
    label='Centroids'
)
plt.title(
    'K-Means Clustering on PCA-Reduced Data'
)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.grid(True)
plt.show()
silhouette_avg_pca = silhouette_score(
    principal_components,
```

```
    clusters_pca
)
print(
    f"\nSilhouette Score for K-Means on "
    f"PCA-Reduced Data (K=4): "
    f"{silhouette_avg_pca:.3f}"
)
```

OUTPUT:


Elbow Method for Optimal K (K-Means)
K-Means Clustering with K=3
PCA - Dimensionality Reduction to 2 Components
K-Means Clustering on PCA-Reduced Data

RESULT:

The K-Means clustering and Principal Component Analysis (PCA) techniques were successfully implemented on the given dataset.

- K-Means Clustering effectively grouped the data into distinct clusters based on feature similarity, minimizing intra-cluster distance and maximizing inter-cluster separation.
- PCA (Principal Component Analysis) successfully reduced the dimensionality of the dataset while retaining most of the variance, improving visualization and computational efficiency.

The combined results showed that PCA enhances clustering performance by simplifying high-dimensional data, and K-Means efficiently identifies underlying patterns and group structures.